

AG para otimização de problemas complexos

- Problemas com restrições
- Problemas multi-objetivos
- Problemas Multimodais
- Micro-AG
- AG paralelos

AG em Problemas com Restrições

Em computação evolutiva, a ligação do problema com o algoritmo é feita na **função de avaliação**, que dá uma nota ao indivíduo.

Melhores indivíduos tem maiores chances de sobrevivência e reprodução.

Portanto, é essencial que a função de avaliação defina o problema de forma perfeita.

Muitas vezes, uma população contém **indivíduos viáveis e inviáveis**.

Deve-se procurar uma avaliação apropriada para estes indivíduos, já que irá influenciar diretamente os resultados do algoritmo.

Problemas com Restrições

A tarefa de processar indivíduos inviáveis é muito importante em problemas de otimização com restrições.

As soluções do problema devem satisfazer certas restrições

- Em GAs, técnicas de inicialização e operadores específicos garantem genes dentro do domínio.
- Isso não garante que o cromossomo satisfaça outras restrições.

Exemplos :

- normas técnicas
- limites técnicos ou físicos
- Conhecimento do especialista

Problemas com Restrições

Restrições:

Funções de igualdade ou desigualdade descrevem os limites desejáveis de projeto.

Podem atuar diretamente sobre as variáveis do problema:

- Limitando seus valores: **restrições laterais**
- Expressando uma condição desejável: **restrições de comportamento**

Problemas com Restrições

Para um problema contínuo, deseja-se encontrar x de forma a:

Minimizar / Maximizar: $f(x_1, x_2, \dots, x_n) \in \mathbb{R}^n$; onde $x \in F$

O espaço de busca $S \subseteq \mathbb{R}^n$ e ;

o espaço de busca das soluções viáveis $F \subseteq S$

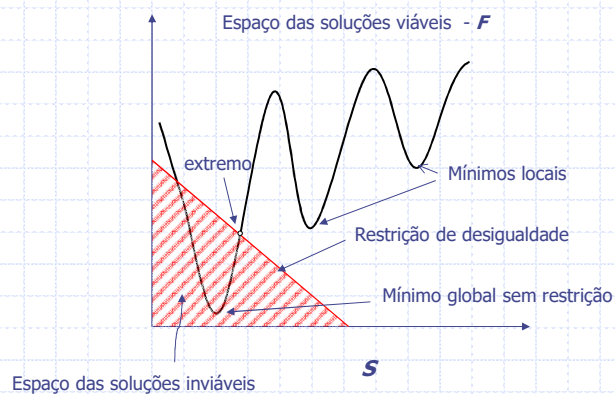
O espaço de busca S é usualmente definido por um retângulo n -dimensional em $\mathbb{R}^n$, e o domínio das variáveis é definido pelos seus limites inferior (l) e superior (u):

$$l(i) \leq x_i \leq u(i) \quad i = 1, 2, \dots, n \quad (\text{restrições laterais})$$

Problemas com Restrições

Minimizar : $f(x) \in \mathbb{R}^n$ $x_L \leq x \leq x_R$

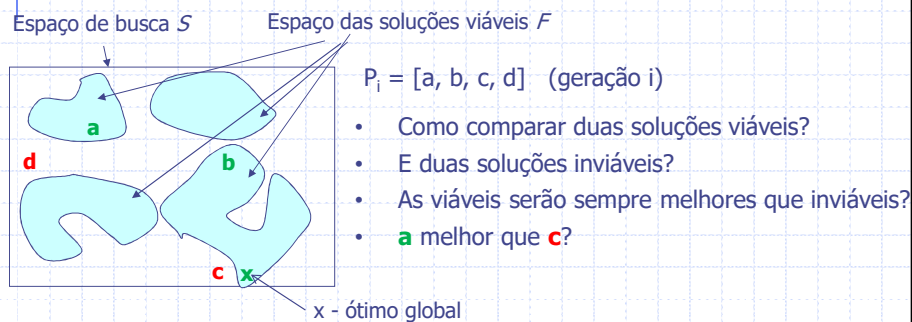
sujeito a: $g(x) \geq A x + B$



Problemas com Restrições

O conjunto das soluções viáveis F é definido pela interseção de S e um conjunto de m restrições adicionais (restrições de comportamento):

$$\begin{aligned} g_i(x) &\geq 0 && \text{para } i = 1, \dots, q \quad \text{e} \\ h_j(x) &= 0 && \text{para } j = q+1, \dots, m \end{aligned}$$



Problemas com Restrições

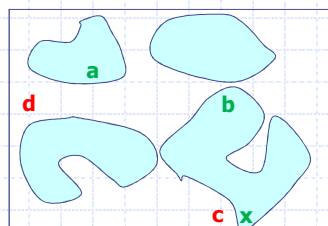
Outras questões:

As soluções inviáveis -

devem ser eliminadas?

devem ser penalizadas? Como? Quanto?

reparadas, movendo-as para pontos mais próximos dentro de F ?



Elitismo em todos indivíduos ou só nos viáveis?

Técnicas para resolver problemas com Restrições

1. Penalização: Função *Penalty* ou Penalidade
 2. Eliminação de Soluções (Rejeição)
 3. Reparo das Soluções
 4. Decodificadores
 5. Técnicas de otimização multiobjetivo
- Etc...

Penalização

É a abordagem mais empregada em AG.

- ◆ Problemas de otimização com restrições podem ser transformados em problemas sem restrições, através do uso de funções de penalidade, associadas à função objetivo, formando uma **única função a ser otimizada**.
- ◆ **A Função Aptidão** é uma composição da função objetivo, com funções de penalidade estabelecidas a partir das restrições do problema

Penalização: Funções Penalidade

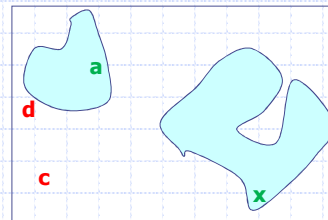
- ◆ Geram-se soluções aleatoriamente sem considerar as restrições
- ◆ Soluções que violam restrições são penalizadas
 - Aptidão (x) = f(x) + Pen(x) se problema de minimização
 - Aptidão (x) = f(x) - Pen(x) se problema de maximização
 - Pen(x) = 0 para todas as soluções viáveis
 - Pen(x) > 0 para todas as soluções inviáveis

Geralmente, a parcela "Pen(x)" é representada por um somatório de funções de penalidades diferentes, e f(x) é a função objetivo do problema.

$$\text{Pen}(x) = \sum_{i=1, \dots, q} r_i \cdot G_i + \sum_{j=q+1, \dots, m} c_j \cdot H_j$$

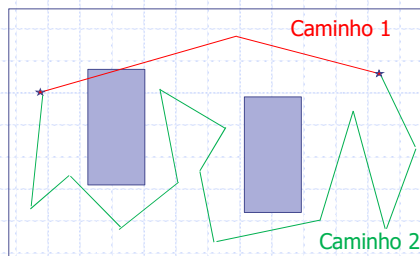
Penalidades

- ◆ Como empregar as funções de penalidade?
- ◆ Pode ser difícil para o AG obter sucesso se penalidades forem muito altas ou muito baixas:
 - Se pen altas e o ótimo está na fronteira da região viável o AG irá rapidamente para a região viável - não explorando a região inviável desde o início do processo de busca
 - Para 2 regiões viáveis separadas, AG pode mover **c** ou **d** para a região que não contém o ótimo global e ficar preso nela.
 - Se pen muito baixas o AG gastará muito tempo explorando a região inviável (penalidade desprezível em relação a função objetivo)
- ◆ Estas questões são importantes porque em grande parte dos problemas o ótimo se localiza na fronteira entre a região viável e a inviável.



Penalidades

Exemplo:



O Caminho 1 é penalizado:

$$\text{Aptidão}(x) = f(x) + \text{Pen}(x)$$

O melhor indivíduo inviável é **pior** que o pior indivíduo viável?

O Caminho 2 merece uma avaliação melhor que o caminho 1?

Penalidades

◆ Existem ao menos 3 escolhas para se definir a relação entre um indivíduo não viável e a região viável do espaço de busca:

1. Um indivíduo pode ser penalizado somente por ser inviável (sem usar informação de sua proximidade com região viável)
2. Penalizado pela "quantidade" que viola uma restrição
3. Pelo esforço de reparar esse indivíduo (custo para torná-lo viável)

Penalidades

- ◆ As funções de penalidade num problema com restrições, podem empregar:
 - ◆ penalidades estáticas,
 - ◆ penalidades dinâmicas,
 - ◆ penalidades adaptativas

Outra alternativa é incorporar os fatores de penalidades nas soluções candidatas e variar junto dos componentes de uma solução.

Função Penalidade

- ◆ Tipos de funções com relação ao grau de violação (desvio):

- Logarítmica

$$\text{Pen}(x) = \ln(1 + k \cdot (\text{desvio}))$$
- Linear

$$\text{Pen}(x) = k \cdot (\text{desvio})$$
- Quadrática

$$\text{Pen}(x) = (k \cdot (\text{desvio}))^2$$

$k = \text{constante}$

Penalidades - Exemplo

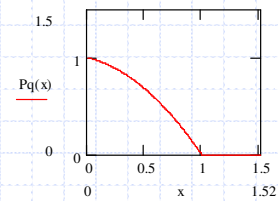
Restrições de tensão: $\left(\frac{1.5 \cdot C_a \cdot C_f \cdot \sigma_y}{\sigma_e} \right)$

Restrições geométricas: $\left(\frac{\alpha_{proj}}{\alpha} \right) \quad \left(\frac{\Delta \alpha_{flex}}{\Delta \alpha} \right)$

Função de penalidades:

$$P_q = \begin{cases} k \cdot [(1 - \tan(\theta)) \cdot x^2 + (\tan(\theta) - 2) \cdot x + 1] & \text{se } x < 1 \\ 0 & \text{se } x \geq 1 \end{cases}$$

Os valores de k podem ser atualizados iterativamente ao longo da busca.



Para $k=1$

Eliminação de Soluções

- ◆ Soluções que violam restrições são simplesmente eliminadas:

$$\text{Aptidão}(x) = 0$$

Indivíduos são rejeitados.

- ◆ Novas soluções têm que ser geradas para completar a população.
- ◆ Muitos dos indivíduos podem ser eliminados causando prejuízos no desempenho do algoritmo
- ◆ AGs combinam informação parcial de toda a população. Assim, uma solução inviável pode prover informação essencial para o processo de evolução.
- ◆ Utiliza-se essa técnica em problemas onde a probabilidade de gerar soluções inválidas pelos operadores genéticos é baixa.
- ◆ Fácil implementação

Reparo das Soluções

- ◆ Soluções que violam restrições são corrigidas por um algoritmo específico. Não existe uma heurística padrão...
- ◆ Um cromossomo viável é gerado a partir de pequenas modificações a um inviável
- ◆ Algoritmos de reparo podem ser complexos ou intensivos computacionalmente.
- ◆ A utilização também depende da probabilidade de geração de soluções que violam restrições.
- ◆ Pode ser aplicado a todos os cromossomos inviáveis ou a um percentual deles.
- ◆ Muito empregado em problemas de otimização combinatória onde a implementação é relativamente simples.

Reparo das Soluções: Busca Local

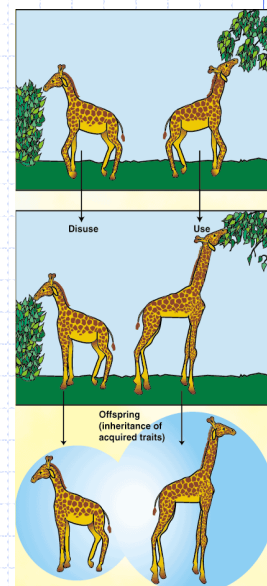
A substituição de indivíduos reparados está relacionada com o **Lamarckismo**:

“Um indivíduo melhora no decorrer da sua vida (aumenta a sua adaptabilidade), o resultado deste processo é codificado de volta no cromossomo.”

É um aprendizado que melhora a *fitness* e é passado aos descendentes.

Este aprendizado que ocorre durante a vida de um indivíduo: **busca local**

“busca local para a solução viável mais próxima”



Reparo das Soluções: Busca Local

Outra forma de interação entre evolução e aprendizado:

Efeito Baldwin

As melhorias não são codificadas de volta no cromossomo, somente o valor da *fitness* é transferido ao indivíduo.

Isto tem um efeito de alterar a superfície de busca, mas a evolução permanece Darwiniana.

Decodificadores

- ◆ Nesta técnica, um cromossomo “dá instruções” de como construir uma solução viável (o cromossomo não codifica diretamente uma solução): Utiliza mapeador da representação que garante a transformação de um indivíduo em uma solução válida.
- ◆ Outras técnicas desenvolvem **operadores genéticos** especialmente para um dado problema.
- ◆ **Decodificadores** e **operadores modificados** podem ser complexos, intensivos computacionalmente, ou mesmo impossível de serem implementados.
- ◆ Quando corretamente projetados, resultam em excelente desempenho, porém exigem um conhecimento amplo do problema aplicado

Separando indivíduos e restrições:

- Função de avaliação f
- Medidas de violação de restrições f_j para m restrições

Uso de métodos de **otimização multiobjetivo** para minimizar um vetor V de dimensão $(m+1)$:

$$V = (f, f_1, \dots, f_m)$$

Solução ideal:

$$f_j(x) = 0 \quad \text{para } 1 \leq j \leq m \quad \text{e}$$

$$f(x) \leq f(y) \quad \text{para todo } y \text{ viável} \in F \text{ (minimização)}$$

Otimização Multiobjetivo

Muitos problemas reais envolvem múltiplos critérios de avaliação:

Múltiplos objetivos

Estes objetivos são muitas vezes conflitantes.

- Exemplos:
 - ♦ Para um produto comercial, minimizar o custo e maximizar a qualidade
 - ♦ Para um projeto, minimizar a área ocupada e maximizar a confiabilidade

Otimização Multiobjetivo

- Otimizar cada objetivo separadamente não produz bons resultados:
A otimização de um único objetivo pode levar os demais objetivos a não serem satisfeitos.
- Técnicas tradicionais de otimização, como *Simulated Annealing*, Gradiente Descendente e Algoritmos Genéticos exigem uma avaliação global única

Otimização Multiobjetivo

Como comparar soluções?

- ◆ Solução 1
 - Objetivo 1: 52% satisfeito
 - Objetivo 2: 28% satisfeito
- ◆ Solução 2
 - Objetivo 1: 35% satisfeito
 - Objetivo 2: 55% satisfeito
- ◆ Qual deve ser a melhor escolha?

Otimização Multiobjetivo

Uma estratégia consiste em combinar as avaliações de cada objetivo para obter uma avaliação global para cada solução:

$$\text{Aptidão} = F(f_1, f_2, \dots, f_n)$$

Métodos de Avaliação

- Agregação de Objetivos
- Estratégia de Pareto
- Outros...

Otimização Multiobjetivo

Agregação de Objetivos

- ◆ É uma abordagem simples que agrega todos os objetivos numa nova função obtida da soma ponderada das funções objetivos

$$F = \sum w_i \cdot F_i \quad i = 1, n$$

onde w_i representa o peso dado para cada objetivo

- ◆ Ex: Se os objetivos tiverem o mesmo peso, $w_i=1$:

- Solução 1: $52+28 = 80$
- Solução 2: $35+55 = 90$

Otimização Multiobjetivo: Agregação de Objetivos

◆ Vantagens

- Simplicidade de implementação
- Pode ser usada em técnicas tradicionais de otimização

◆ Desvantagens

- Fraca performance na prática: exige aplicação de conhecimentos específicos do problema para o ajuste ótimo dos pesos w_i
- Requer que as avaliações sejam normalizadas para a soma de valores comparáveis entre si

Otimização Multiobjetivo: Estratégia de Pareto

Vilfredo Pareto: um dos primeiros a empregar esta técnica

◆ Compara soluções sem combinar as avaliações

◆ Conceito de dominação e não-domação:

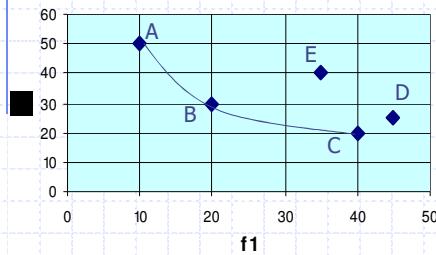
Num problema de minimização, uma solução x é dita **não dominada** por outra solução y se:

- Para todos os objetivos é verificado que a avaliação de x seja menor ou igual que a de y
- Existe pelo menos um objetivo onde é verificado que a avaliação de x é menor a de y

$$(\forall i: f_i(x) \leq f_i(y)) \wedge (\exists i: f_i(x) < f_i(y))$$

Otimização Multiobjetivo: Estratégia de Pareto

Exemplo: Produtor deseja minimizar custos de produção (f_1) e distribuição (f_2)



Solução	f_1	f_2
A	10	50
B	20	30
C	40	20
D	45	25
E	35	40

- ✓ A, B e C são boas soluções (embora nenhuma seja melhor em ambos os critérios); A, B e C são não-dominadas porque não há soluções melhores em ambos os critérios.
- ✓ D e E são dominadas por outras soluções: $B < E$ e $C < D$.

Otimização Multiobjetivo: Estratégia de Pareto

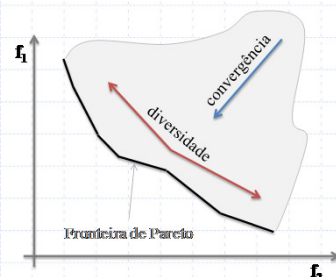
O objetivo do AG é encontrar o conjunto Pareto-ótimo:

Fronteira de Pareto: conjunto de todas as soluções não dominadas do espaço de busca.

Qualquer que seja o critério para a avaliação global, deve ser garantido que a melhor solução faz parte desse conjunto.

Metas que um AG multi-objetivo deve alcançar:

- Convergência - guiar a busca na direção do fronte de Pareto; e
- Diversidade - população distribuída no fronte de Pareto.



Otimização Multiobjetivo: Estratégia de Pareto

- ◆ Evolução é feita privilegiando os indivíduos que não estão dominados por nenhum outro
- ◆ Avaliação final de uma solução pode levar em consideração o número de indivíduos na população que ela domina
- ◆ Vantagens
 - Não exige que as avaliações sejam normalizadas pois não combina avaliações de diferentes objetivos
- ◆ Desvantagens
 - Maior complexidade de implementação
 - Necessita de um critério adicional para a escolha de uma solução ótima dentre as pertencentes ao conjunto

Algoritmos Evolutivos + Estratégias de Pareto

VEGA - Vector Evaluated Genetic Algorithms

Pioneiro na implementação de algoritmos evolutivos para solução de problemas multi-objetivos

Schaffer, J. D., "Multiple Objective Optimization with Vector Evaluated Genetic Algorithms", Ph.D. Thesis, Vanderbilt University, 1984

Extensão do software GENESIS para incluir funções multicritério.

- Divisão da população em subpopulações de tamanhos iguais ($N_{\text{indiv}} / N_{\text{obj}}$)
- Cada subpopulação responsável por um único objetivo.
- Seleção – feita independentemente para cada objetivo
- Cruzamento – sorteado aleatoriamente e realizado entre as subpopulações

Desvantagem: a mistura das sub-populações é equivalente a se utilizar uma combinação linear dos componentes do vetor de aptidão tratando o problema como uni-objetivo.

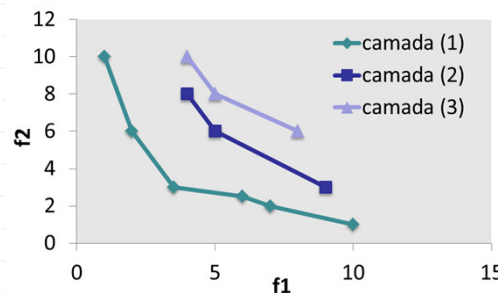
Schaffer chamou este problema de especialização.

Algoritmos Evolutivos + Estratégias de Pareto

Goldberg (1989) – procedimento de ordenamento não-dominado

• A ideia é atribuir uma probabilidade de reprodução igual para todos os indivíduos não-dominados na população.

1. Os indivíduos não-dominados recebem uma posição ou ordem 1
2. Indiv de ordem 1 são removidos da população e reservados.
3. Novo conjunto de indivíduos não dominados: atribui-se a ordem 2 e assim por diante.



Outros Algoritmos Evolutivos Multiobjetivos

Nos últimos anos muitos pesquisadores têm modificado as idéias iniciais propostas por Goldberg para tratamento de problemas multiobjetivos.

Assim, atualmente existem inúmeras implementações.

Principais implementações:

- **MOGA** - Fonseca, C. M. and Fleming, P.J., "Genetic Algorithms for Multi-Objective Optimization: Formulation, Discussion and Generalization", Proc. Fifth Int. Conf. on Gen. Alg., pp 416-423, 1993.
- **NSGA** - Srinivas, N. And Deb, K., "Multiobjective Optimization Using Non-dominated Sorting in Genetic Algorithm", Evolut. Comput., 2(3) 221-248, 1994.
- **NSGA-II** - Deb, K., Agrawal, S., Pratap, A. & Meyarivan, T., "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II". In: Technical Report 200001, Indian Institute of Technology, Kanpur, 2000.

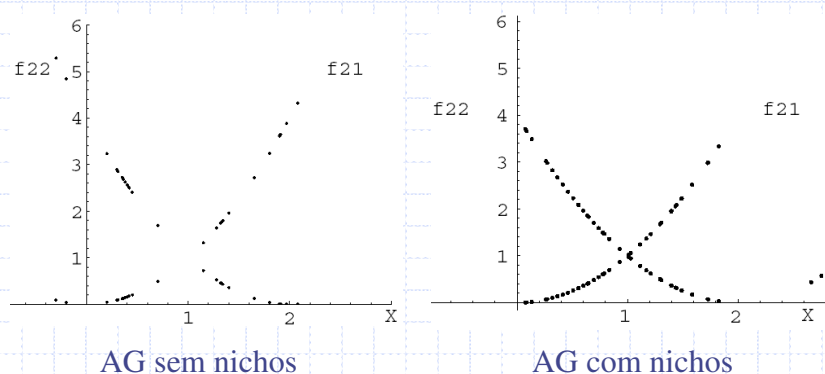
MOGA - Fonseca e Fleming

Multi-objective Optimization Genetic Algorithm (MOGA)

- Usa o procedimento de ordenamento não-dominado de Goldberg.
- Os indiv. dominados recebem uma ordem igual a 1 + número de indivíduos da população pelos quais ele é dominado.
- A rotina de seleção usa este ordenamento para selecionar os indivíduos para reprodução
- A atribuição de aptidão por camadas, colocando no mesmo nível indivíduos de mesmo grau de dominância, leva a produzir uma grande pressão de seleção que pode causar convergência prematura.
- Preocupados com a manutenção da diversidade, foi proposto o uso do método de formação de nichos (compartilhamento da aptidão) para distribuir a população ao longo da região ótima de Pareto.
- **Desvantagem:** lenta convergência e dificuldade de escolha do tamanho do nicho.

MOGA - Fonseca e Fleming

Multi-objective Optimization Genetic Algorithm (MOGA)



NSGA - Srinivas e Deb

Nondominated Sorting Genetic Algorithm (NSGA)

- Baseado em camadas de classificações de indivíduos
- Antes da seleção a pop. é ordenada de acordo com não-dominância:
 - Todos indiv. não dominados são classificados num nível com aptidão fictícia alta
 - Para manter a diversidade, estes indiv fazem o compartilhamento da aptidão
- Então este grupo é separado e cria-se outra camada de indiv não dominados que recebem uma aptidão um pouco menor.
- O processo continua até que todos os indiv da população sejam classificados.
- A reprodução da população é efetuada utilizando-se a aptidão compartilhada.

Como o primeiro nível de soluções não-dominadas possui as mais altas aptidões, um maior número de cópias dos seus indivíduos será realizado e levará a busca para a fronteira de Pareto.

NSGA - Srinivas e Deb

NSGA-I

- O algoritmo, de maneira geral, possui uma rápida convergência.
- Como ponto fraco apresenta novos parâmetros com forte influência no desempenho da busca, especialmente o tamanho do nicho.

NSGA-II

Modificações no NSGA:

- procedimento de ordenamento rápido de soluções não-dominadas,
- mecanismo de elitismo,
- operador de aglomeração sem parâmetros de ajuste para o manutenção da diversidade no conjunto de soluções externas.

Este algoritmo é considerado como o de melhor desempenho.

AG em Problemas Multimodais

Nichos

Na natureza, nichos são subespaços do ambiente na qual uma **variedade de espécies** compete por uma quantidade de recursos disponíveis e muitas vezes limitada.

Nos AGs tradicionais esta **variedade** de espécies tende a diminuir e os indivíduos da população convergem rapidamente para um único ponto.

As vantagens de uma população de soluções (**diversidade**) diminuem quando todas as soluções são iguais.

Mesmo quando existem múltiplas soluções interessantes para um determinado problema, o AG clássico localiza apenas uma delas.

Nichos

◆ Nos AGs, emprega-se os nichos para:

- Promover a formação de subpopulações estáveis na vizinhança das soluções ótimas globais e locais
- Melhorar a diversidade
- Permitir o AG localizar múltiplas soluções ótimas
- Evitar o *genetic drift*

Nichos

Algumas causas da redução de diversidade da população:

- Seleção elimina indivíduos com aptidão reduzida e privilegia aqueles com aptidão elevada (pressão da seleção).
- Superindivíduos

Deve-se empregar técnicas para manter a diversidade nas populações.

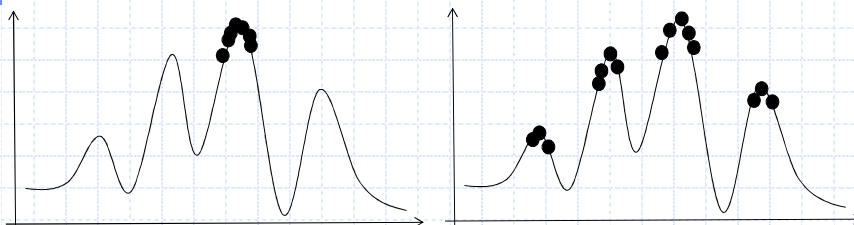
Métodos de *niching* mais empregados :

Algoritmo de **Compartilhamento de Aptidão** (*fitness sharing*) e **Crowding**

Nichos

Compartilhamento de Aptidão (Goldberg e Richardson, 1987)

Esquema prático para solucionar o problema de otimização de funções multimodais.



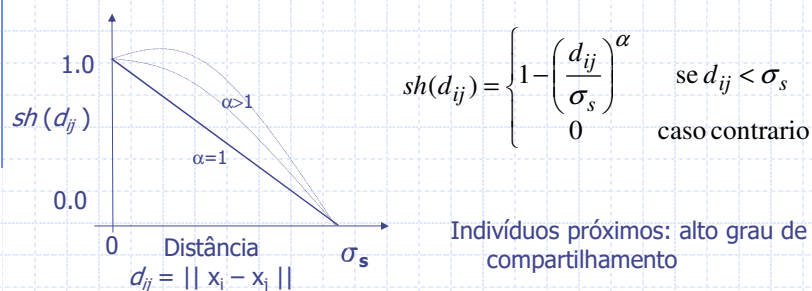
Compartilhamento de Aptidão

Objetivo:

- Reduzir a aptidão dos indivíduos nas regiões do espaço de busca densamente povoado.
- A aptidão de um indivíduo é reduzida proporcionalmente à quantidade de indivíduos similares a ele .
- Uma função é definida para determinar a **vizinhança** e o **grau de compartilhamento** para cada indivíduo da população.

Compartilhamento de Aptidão

Função de Compartilhamento (Goldberg 1989)



σ_s é o raio do nicho e definido pelo usuário.

α é uma constante (geralmente 1) capaz de regular a forma da função de compartilhamento.

Indivíduos fora do raio do nicho não compartilham aptidão (considerados diferentes).

Compartilhamento de Aptidão

- Indivíduos próximos a outro indivíduo apresentam um alto grau de compartilhamento: próximo a 1.0.
- Indivíduos menos similares (longe) apresentam um grau muito pequeno de compartilhamento: próximo a 0.0.
- Se um indivíduo é idêntico a outro, seu grau de compartilhamento será igual a 1.0.

Compartilhamento de Aptidão

Para um determinado indivíduo o grau de compartilhamento é determinado somando-se os valores da **função sh** contribuídos por todos os outros indivíduos na população.

Após acumular os valores da função de compartilhamento, o indivíduo que está sendo avaliado terá **seu valor de aptidão reduzido**.

O **compartilhamento reduz a aptidão** de um indivíduo proporcionalmente à **quantidade de indivíduos similares** a ele dentro da população.

$$f_s(x_i) = \frac{f(x_i)}{\sum_{j=1}^N sh(d(x_i, x_j))}$$

$f_s(x_i)$ – aptidão compartilhada

$f(x_i)$ - valor de aptidão

$sh(x)$ - função de compartilhamento

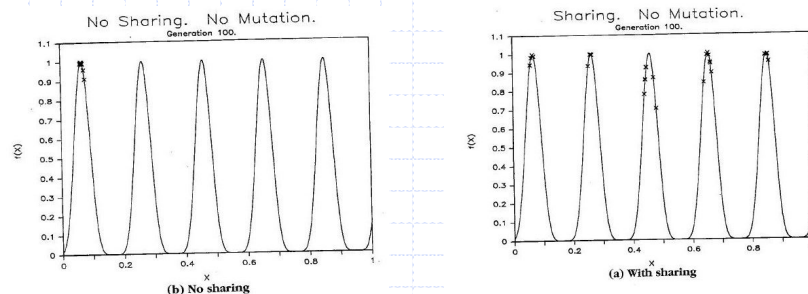
N é a quantidade de indivíduos da população

Compartilhamento de Aptidão

- ◆ Como resultado, este mecanismo limita o crescimento descontrolado de espécies particulares dentro de uma população.
- ◆ O algoritmo permite uma menor probabilidade de que muitos indivíduos do mesmo nicho sejam selecionados, forçando uma maior diversidade populacional.
- ◆ Características do algoritmo:
 - Aumento do custo computacional
 - Tamanho da população deve ser proporcional ao número de ótimos

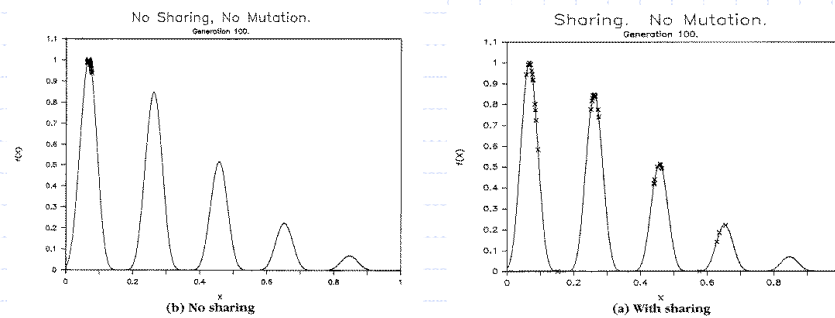
Compartilhamento de Aptidão

Exemplo:
Distribuição da população na 100a. geração



Goldberg e Richardson (1987)

Compartilhamento de Aptidão



Nichos

Outra técnica empregada para manter a diversidade é:

Crowding

Inspiração: indivíduos similares deverão competir entre si.

1a. concepção foi proposta por *De Jong* em 1975 para manter a diversidade da população

Mahfoud (1995) propôs uma variante:

A substituição de indivíduos na população é modificada:

novos indivíduos substituem outros indivíduos similares com menor valor de aptidão

A similaridade pode ser obtida através de medidas de distância genotípica ou fenotípica.

Nichos

Crowding - Idéias básicas:

- Os filhos são geralmente similares aos pais.
- O pai mais similar ao filho de maior aptidão é substituído.

Agrupe todos os indivíduos da população em $N/2$ pares

Cruze todos os pares e faça mutação nos filhos

Escolha aleatoriamente CF pais
(CF *crowding factor*: usualmente 2 ou 3)

Se a aptidão de um dos filhos é maior, então substitua o pai mais parecido numa comparação *bit a bit*.

} substituição

- As técnicas de *crowding* permitem a perda de bons indivíduos, denominada de erro de substituição.

Nichos

Estes métodos de *Niching* são técnicas importantes para a resolução de problemas de otimização multimodais e aprendizado de máquina.

Mahfoud (*Niching Methods for Genetic Algorithms. Ph.D. thesis, University of Illinois at Urbana-Champaign, 1995*):

Compara vários métodos de *niching* que são agrupados: paralelos e sequenciais

Os paralelos formam e mantêm os nichos simultaneamente dentro da população.

Os sequenciais localizam múltiplos nichos temporariamente.

As análises indicam melhores resultados para os nichos paralelos.

Micro-GA

Varição do AG clássico → emprega uma **população reduzida**

A maioria dos AGs não tem bom comportamento com populações pequenas
→ processam informação insuficiente e convergem prematuramente para resultados não ótimos.

No AG clássico a população deve ser numerosa para uma boa exploração do espaço.

População numerosa = alto custo computacional em problemas complexos

Também evita-se a ocorrência de convergência prematura: bons indivíduos tendem a dominar a população e convergir para um ótimo local.

Micro-GA

μGA explora a possibilidade de se empregar populações bem menores que o AG clássico:

- Para não ser enganado por um ótimo local, ele adota uma estratégia que sempre mantém uma diversidade suficiente na população.
- A população é periodicamente renovada sempre que a diversidade cai abaixo de um determinado limite.

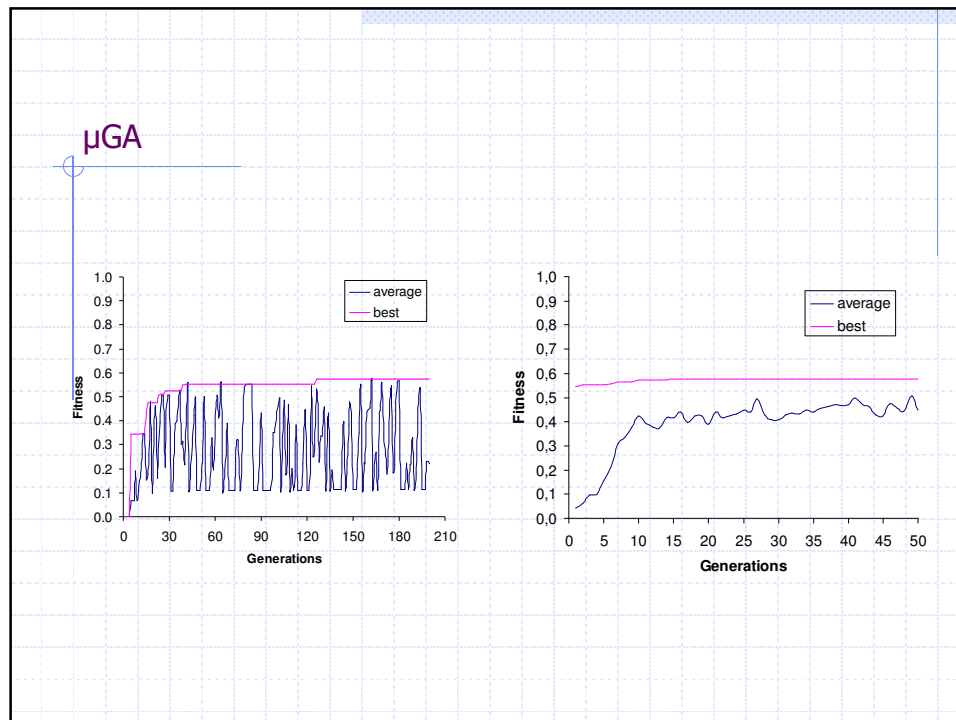
μ GA

Usualmente emprega-se:

- ◆ População ≈ 5 indivíduos
- ◆ Elitismo – melhor indivíduo é transferido para a próxima população (garante que não haverá perda de informação contida num bom cromossomo)
- ◆ Restante da nova população é gerado aleatoriamente
- ◆ Emprega-se seleção por torneio ao invés da roleta. Usualmente toma-se $k = 2$ (pegar 2 indivíduos da população).

μ GA

- ◆ Aplica-se *crossover* com $P_c = 1$ para favorecer a troca de material genético entre a população
- ◆ **Carroll** aplica $P_c = 0.50$ quando emprega *crossover* uniforme e sempre só produz um filho de cada par de indivíduos.
- ◆ Não é necessário o emprego de mutação: $P_m = 0$
(a diversidade é garantida em cada geração pela maioria da população gerada aleatoriamente)



Micro-Ga

Micro-GA tem demonstrado boa performance em termos de número de avaliações quando comparado ao AG clássico.

Referências:

- ◆ Krishnakumar, K., "Micro-Genetic Algorithms for Stationary and Non-Stationary Function Optimization," SPIE: Intelligent Control and Adaptive Systems, Vol. 1196, Philadelphia, PA, 1989.
- ◆ D.L. Carroll's FORTRAN Genetic Algorithm Driver
<http://cuaerospace.com/carroll/ga.html>
<http://www.cuaerospace.com/Technology/Genetic-Algorithm/GA-Driver-Free-Version>

Algoritmos Genéticos Paralelos

Grande vantagem dos AGs : naturalmente bem estruturados para se paralelizar.

É possível dividir suas tarefas entre os processadores de sistemas computacionais paralelos e distribuídos.

Vantagens:

- Fornece amplo poder computacional com melhora na velocidade e no desempenho do AG.

Principais modelos de algoritmos genéticos paralelos (AGP):

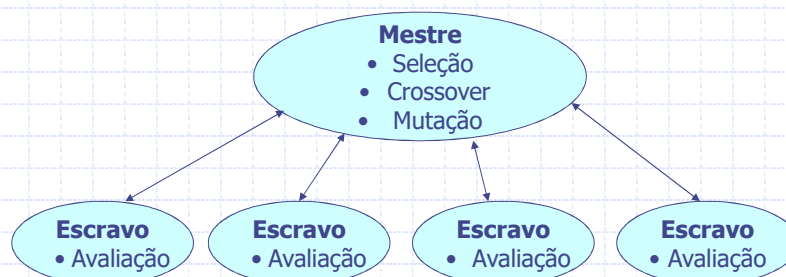
- Global (ou “**mestre- escravos**”)
- AGP com Granularidade baixa (Modelo de ilhas)
- AGP com Granularidade alta

Algoritmos Genéticos Paralelos

♦ **Modelo Global** ou “mestre- escravos”

Uma única população é submetida a operadores de seleção, *crossover* e mutação em um processador principal (mestre).

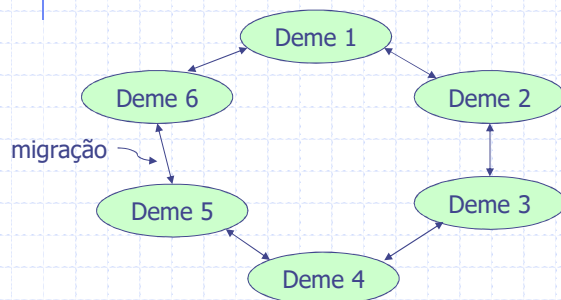
O mestre envia os indivíduos para avaliação nos diversos processadores (escravos), e espera o resultado para prosseguir para as próximas gerações.



Algoritmos Genéticos Paralelos

◆ **Modelo de ilhas** ou AGP com granularidade baixa

Neste modelo a população é dividida em subpopulações (demes: colônias de células). A evolução é isolada, delegando assim a diversos processadores, não só a função de avaliação de indivíduos, mas também as funções de reprodução e seleção destes.



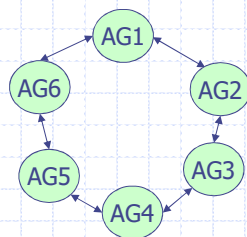
Após um dado número de gerações, os demes trocam entre si os melhores indivíduos e continuam a evolução isoladamente (**migração**).

Este processo de troca de indivíduos é repetido a cada número de gerações pré-determinado

Algoritmos Genéticos Paralelos: Topologia das ilhas

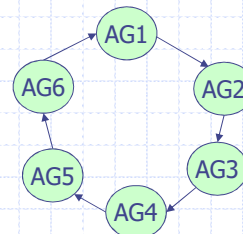
■ Migração de vizinhança

- ◆ Bi-direcional

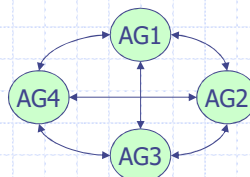


■ Migração em anel

- ◆ Indiv migram direcionalmente entre ilhas adjacentes



■ Migração irrestrita



Algoritmos Genéticos Paralelos

O **modelo de ilhas** é o mais utilizado [Lacerda, 1999 e Cantú-Paz, 1997].

Sua implementação é simples:

Rodar cada AG convencional, responsável por uma sub-população, em um processador diferente, fazendo a migração de tempos em tempos (Melhora a diversidade nas subpopulações).

Este modelo de paralelização pode ser implementado numa rede de computadores ou até mesmo num simples computador. Basta adicionar algumas subrotinas para migração no AG convencional.

Algoritmos Genéticos Paralelos

◆ Parâmetros do modelo de ilhas

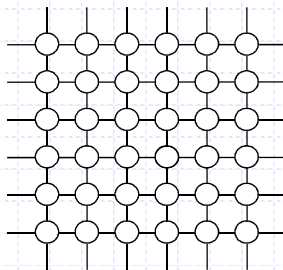
- Tamanho das subpopulações
- Frequência das ocorrências de migração.
- Número de indivíduos imigrantes (em geral eles substituem os piores indivíduos da subpopulação destino)
- Topologia da rede ou dos processadores

Algoritmos Genéticos Paralelos

◆ AGP com granularidade alta

Cada indivíduo é colocado em um processador, geralmente arrumados numa malha 2-D de processadores.

A seleção é aplicada em cada processador, entre ele e os processadores vizinhos.



O *crossover* é realizado localmente, apenas com indivíduos de processadores vizinhos.

Algoritmos Genéticos Paralelos

Dos três modelos apresentados, o modelo mestre-escravos é o único que não modifica o funcionamento do AG tradicional.

O mesmo não ocorre com outros modelos, porque o *crossover* não pode ser realizado com qualquer indivíduo da população.

Pode-se construir um algoritmo genético paralelo misto, onde se combinam esses métodos.

Por exemplo, pode-se implementar um modelo de ilhas, onde cada ilha executa um algoritmo genético com granulação fina, ou que cada ilha evolua usando um algoritmo do tipo "mestre-escravos".

Sítios interessantes:

- **ILLiGAL (Illinois Genetic Algorithms Laboratory)**
David E. Goldberg - University of Illinois
Vários códigos disponíveis por ftp (C e pascal):
AGS e LCS (Sist. Classificadores):

<https://illigal.org/>

- Código em C do programa GENESIS de John Grefenstette ,
a partir do: **GA Archive**

<http://www.aic.nrl.navy.mil/galist/>

- Código em C++ **Open BEAGLE** (Université Laval, Québec, Canada)

<http://beagle.gel.ulaval.ca/index.html>

<https://github.com/chgagne/beagle>

Cont. sítios interessantes:

- **D.L. Carroll's FORTRAN Genetic Algorithm Driver**
<http://cuaerospace.com/carroll/ga.html>

- **Genetic Java!!! Genetic Algorithm Applets**

<http://geneticalgorithms.ai-depot.com/Programs.html>

- **Genetic Algorithm Experiment**

<http://www.aai.org/AITopics/html/genalg.html>

<https://aitopics.org/search?filters=&sort=score+desc&q=genetic+algorithm>

<http://www.rennard.org/alife/english/gavgb.html>